pcap_dispatch is identical in usage. The only difference between *pcap_loop* and *pcap_dispatch* is that *pcap_dispatch* will only process the first batch of packets it receives from the system, while *pcap_loop* will keep processing packets until the count runs out.

The *callback* function should have the signature, which should be as follows:

```
void callback_function(u_char *arg, const struct pcap_pkthdr
*pkthdr,const u_char *packet)
```

The arguments to the *callback* function are as follows:

- arg - This is the list of user parameters you passed to *pcap_loop*
- pkthdr—This parameter contains information about the packet captured. The structure of the *pcap_pkthdr* is given below:

```
struct pcap_pkthdr{
     struct timeval ts;
     bpf_u_int32 caplen;
     bpf_u_int32 len;
}
```

- packet – This is a pointer to the first byte of data containing the entire packet. As you might imagine, the packet isn't a string, but rather a collection of structures.

Now that we have established the basics, we can proceed to build a small packet sniffer using libPCAP.

The code for the packet sniffer is given below:

```
#include <pcap.h>
#include <stdio.h>
#include <stdlib.h>
void callback_function(u_char *arg, const struct pcap_pkthdr*
pkthdr,const u_char* packet)
{
    int i=0;
    static int count=0;
    printf("Count: %d\n", ++count);
    printf("Size: %d\n", pkthdr->len);
    printf("Payload:\n");
    for(i=0;i<pkthdr->len;i++) {
        if(isprint(packet[i]))
            printf("%c ",packet[i]);
        else
            printf(" . ",packet[i]);
        if((i%16==0 && i!=0) || i==pkthdr->len-1)
            printf("\n");
    }
}
int main(int argc,char **argv)
{
    int i;
```

Figure 1: Output

```
    char dev[]="wlan0";
    char errbuf[PCAP_ERRBUF_SIZE];
    pcap_t* descr;
    const u_char *packet;
    struct pcap_pkthdr hdr;
    struct ether_header *eptr;    /* net/ethernet.h */
    struct bpf_program fp;        /* hold compiled program */
    bpf_u_int32 maskp,netp;             /* subnet mask */
    pcap_lookupnet(dev, &netp, &maskp, errbuf);
    descr = pcap_open_live(dev, BUFSIZ, 1,-1, errbuf);
    pcap_compile(descr, &fp, "port 80", 0, netp);
    pcap_setfilter(descr, &fp);
    pcap_loop(descr, -1, callback_function, NULL);
    return 0;
}
```

In the *callback_function*, we are only displaying the packet number, size of the packet and the packet payload. The output of running the above code is given in Figure 1.

With this, I conclude the introduction to programming with libPCAP. You have learned some of the basic concepts behind network sniffing. Do test and experiment with libPCAP, to understand its full potential. There are also other sniffers you could experiment with, namely tcpdump and wireshark. END

By: Sahil Chelaramani

The author is an open source activist, who loves Linux, Python and Android. He is a fourth year student at the Goa Engineering College (GEC). In the author's own words, "Beyond anything else, I love to code."